

Cahier des charges

PR-Review-Bot

Application web de revue automatisée de Pull Requests

Contents

1	Introduction et objectifs	2
2	Liste des fonctionnalités	3
2.1	Intégration et déclenchement	3
2.2	Analyses	3
2.3	Restitution des résultats	4
2.4	Dashboard et suivi	4
3	Schémas d'interface graphique	5
3.1	Écran d'accueil (Home) — connexion à GitHub/GitLab/Bitbucket	5
3.2	Écran de gestion des dépôts (repos)	5
3.3	Écran de revue d'une Pull Request	6
4	Schéma de la base de données	7
4.1	Détail des entités et de leurs champs	7
4.2	Relations et cardinalités	8
5	Planning de développement (jusqu'à la bêta)	9

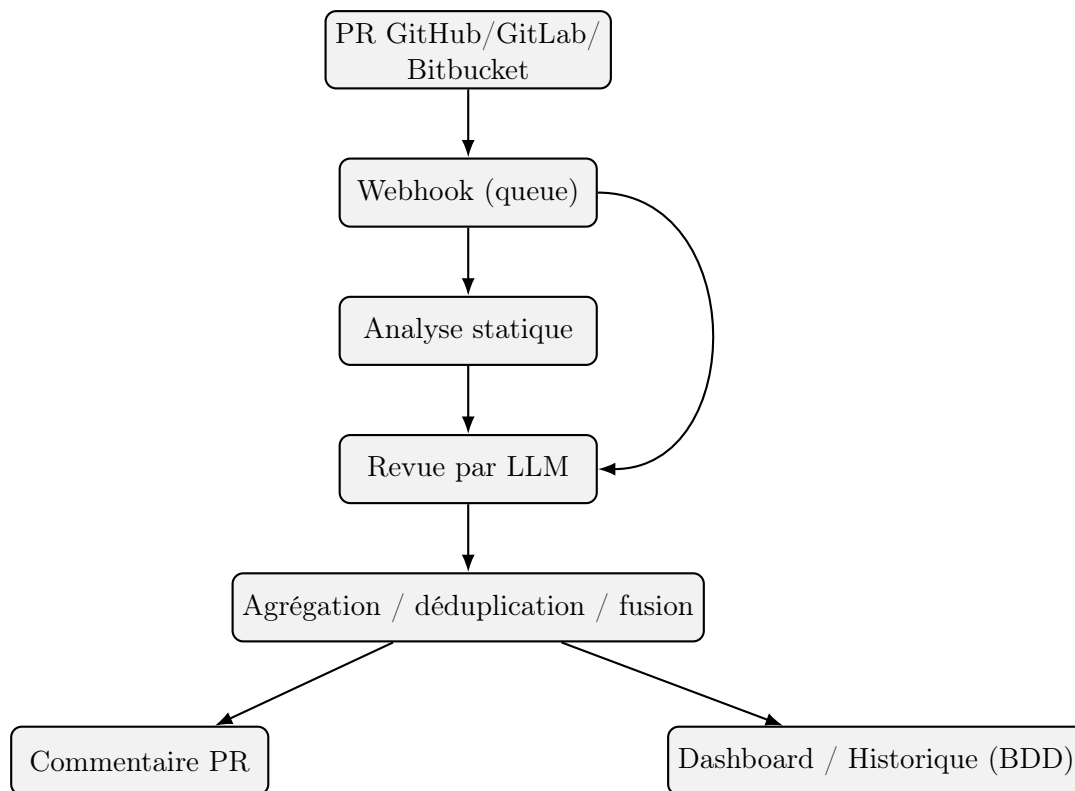
1 Introduction et objectifs

Le projet consiste à développer une application web capable de se connecter à un dépôt hébergé sur **GitHub**, **GitLab** ou **Bitbucket** afin d'y déclencher automatiquement une revue de code sur chaque *Pull Request* (PR). L'application combine des **analyses statiques** (Checkstyle, PMD, SpotBugs, etc.) et une **revue par un LLM**, fusionne les résultats, puis publie un verdict sous forme de commentaires directement sur la PR. Un **dashboard web** permet ensuite de consulter l'historique des revues, de filtrer les problèmes détectés et de les corriger manuellement (flag humain) si l'analyse automatique s'est trompée.

Stack technique retenue

Composant	Technologie
Backend	Micronaut (Java) + Maven
Frontend	Svelte + Bootstrap (UI)
Base de données	DuckDB (persistance via JDBC)
Intégration	Webhooks GitHub / GitLab / Bitbucket

Vue d'ensemble du pipeline



2 Liste des fonctionnalités

2.1 Intégration et déclenchement

F1 — Connexion à un dépôt Git (GitHub / GitLab / Bitbucket)

L'utilisateur authentifie l'application auprès d'un des trois fournisseurs (OAuth ou token d'accès personnel) depuis l'écran d'accueil. Cette connexion permet ensuite de lister et de sélectionner les dépôts sur lesquels activer la revue automatique. C'est le point d'entrée obligatoire avant toute autre fonctionnalité.

F2 — Gestion des dépôts suivis (page "repos")

Une page dédiée liste tous les dépôts connectés à un client, avec leur plateforme, leur nom, leur URL et leur branche par défaut. L'utilisateur peut y activer/désactiver la revue automatique par dépôt, ou en supprimer un. C'est aussi depuis cette page que l'on accède au fichier de configuration `.reviewbot.yml` du dépôt.

F3 — Auto-déclenchement via Webhook

À chaque ouverture ou mise à jour d'une PR sur un dépôt suivi, un webhook envoyé par la plateforme Git notifie notre backend Micronaut. L'évènement est placé dans une file d'attente (queue) pour être traité de façon asynchrone, ce qui évite de bloquer l'API du fournisseur et permet d'absorber les pics de charge.

2.2 Analyses

F4 — Analyse statique du code

Une série d'outils Java est exécutée sur le diff de la PR : Checkstyle (style), PMD (bonnes pratiques), SpotBugs (bugs potentiels), Error Prone (erreurs de compilation subtiles), JaCoCo (couverture de tests), Javadoc (documentation) et JUnit (exécution des tests). Chaque outil produit un rapport structuré (type, sévérité, fichier, ligne).

F5 — Exécution en sandbox

Toutes les analyses (statiques et tests) s'exécutent dans un environnement isolé (conteneur Linux) afin de ne jamais exécuter du code non fiable sur l'infrastructure principale. Le sandbox impose des limites de temps et de ressources (CPU/RAM) et est détruit après chaque analyse.

F6 — Fichier de configuration par dépôt (`.reviewbot.yml`)

Chaque dépôt peut contenir un fichier `.reviewbot.yml` à la racine permettant d'activer/désactiver certaines analyses, d'exclure des répertoires ou fichiers de la revue (ex. dossiers générés, vendor), de fixer des seuils de sévérité, et de personnaliser le comportement du bot pour ce dépôt précis.

F7 — Revue par LLM

En complément des analyses statiques, le diff de la PR est envoyé à un LLM avec un prompt structuré afin de détecter des problèmes moins mécaniques : failles de sécurité, mauvaise gestion des erreurs, qualité de la documentation, cohérence du code avec son contexte. Le LLM retourne des commentaires structurés (fichier, ligne, catégorie, sévérité, explication).

F8 — Agrégation, déduplication et fusion

Les résultats des analyses statiques et de la revue LLM sont fusionnés en un seul jeu de commentaires. Les doublons (même ligne, même type de problème signalé par deux outils différents) sont détectés et fusionnés. Un verdict global de sévérité (bloquant / avertissement / info) est calculé à partir de l'ensemble des remontées.

2.3 Restitution des résultats

F9 — Commentaire global sur la PR

Un commentaire unique est posté en tête de la PR, résumant le verdict global (approuvé / à corriger / bloqué), le nombre de problèmes par catégorie et par sévérité, ainsi qu'un lien vers le dashboard pour le détail complet.

F10 — Commentaires en ligne sur les diffs

Pour chaque problème détecté et localisé, un commentaire est posté directement sur la ligne concernée du diff de la PR, avec sa catégorie (bug, style, sécurité, couverture de tests, documentation) et sa sévérité, à la manière d'une relecture humaine classique.

F11 — Écran de revue de PR (Files / Diff / Comments)

Une page web dédiée permet de visualiser une PR précise : liste des fichiers modifiés à gauche, diff détaillé au centre, et panneau des commentaires (statiques + LLM) à droite. C'est la vue de détail utilisée aussi bien pendant la revue automatique que lors d'une relecture humaine a posteriori.

2.4 Dashboard et suivi

F12 — Dashboard Web (historique des revues)

Une interface centralisée liste l'historique de toutes les revues effectuées, tous dépôts confondus, avec pour chacune la date, le dépôt, la PR concernée, le verdict et le nombre de problèmes détectés.

F13 — Filtres par catégorie et sévérité

Le dashboard permet de filtrer l'historique des revues et des commentaires par catégorie (bug, style, sécurité, tests, documentation) et par niveau de sévérité (bloquant, avertissement, information), afin de retrouver rapidement les problèmes pertinents sur un grand nombre de PR.

F14 — Flag humain

Un humain peut marquer un commentaire automatique comme non pertinent (faux positif, "n'importe quoi") directement depuis le dashboard ou l'écran de revue. Ce retour est conservé en base et pourra servir de base à l'amélioration future du filtrage des résultats.

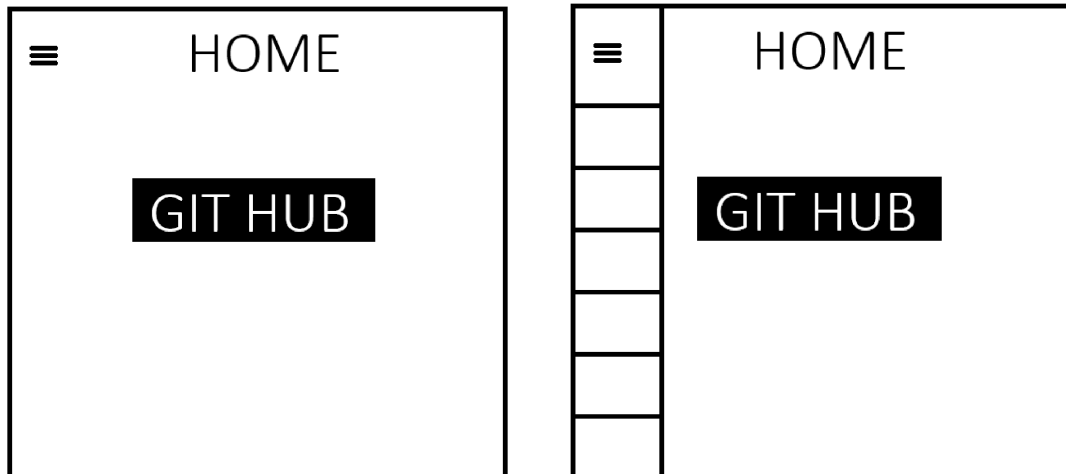
F15 — Persistance des données

L'ensemble des données (clients, dépôts, PR, fichiers, lignes modifiées et commentaires) est persisté dans une base DuckDB via JDBC, ce qui permet de conserver l'historique complet des revues et d'alimenter le dashboard sans dépendre des API des plateformes Git.

3 Schémas d'interface graphique

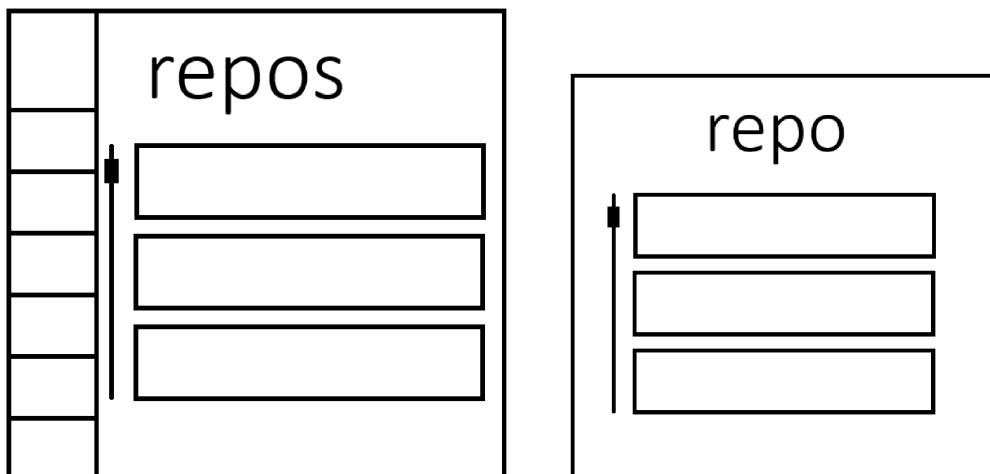
Les maquettes ci-dessous représentent les trois écrans principaux de l'application : la page d'accueil (connexion au fournisseur Git), la page de gestion des dépôts, et la page de revue d'une Pull Request.

3.1 Écran d'accueil (Home) — connexion à GitHub/GitLab/Bitbucket



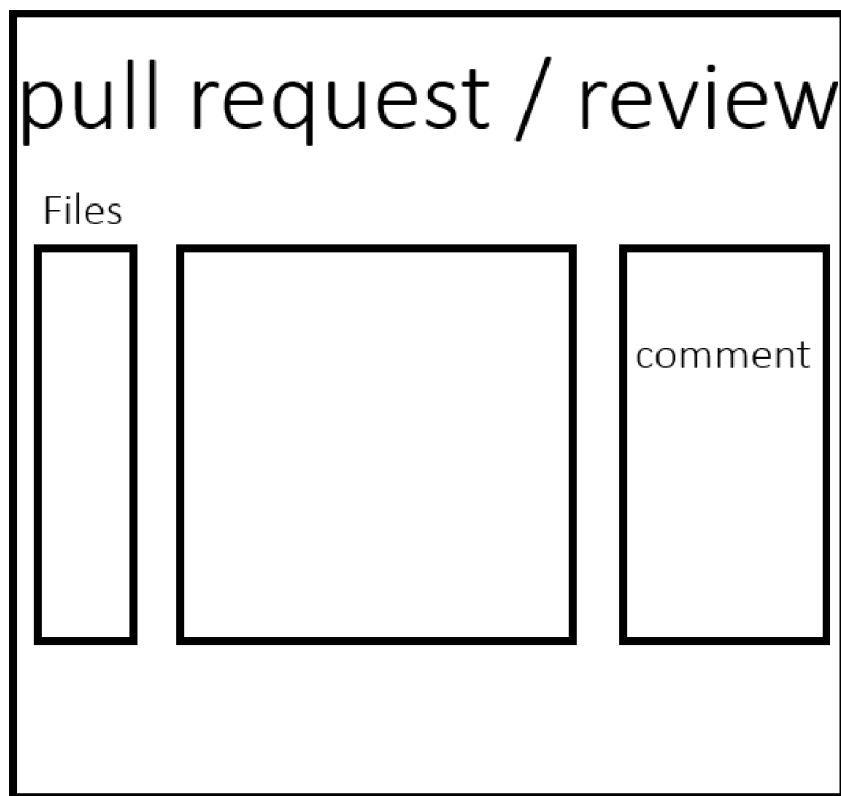
Cet écran est le point d'entrée de l'application. Le menu hamburger (en haut à gauche) ouvre la barre de navigation latérale (visible sur la seconde maquette), qui donne accès aux autres sections une fois connecté. Le bouton central "GIT HUB" permet de lancer l'authentification auprès du fournisseur choisi (GitHub, GitLab ou Bitbucket selon la configuration).

3.2 Écran de gestion des dépôts (repos)



Une fois connecté, l'utilisateur accède à la liste des dépôts disponibles sur son compte. Chaque ligne représente un dépôt (nom, plateforme, branche par défaut) et permet d'activer la revue automatique ou d'accéder à sa configuration (`.reviewbot.yml`). La barre latérale reste visible pour naviguer vers le dashboard ou revenir à l'accueil.

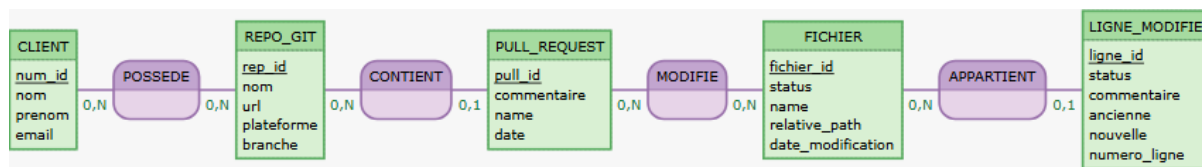
3.3 Écran de revue d'une Pull Request



Cet écran affiche le détail d'une PR : la colonne *Files* liste les fichiers modifiés, la zone centrale affiche le diff du fichier sélectionné avec les annotations issues des analyses statiques et du LLM, et la colonne *comment* regroupe les commentaires (globaux et en ligne) ainsi que la possibilité de les flaguer manuellement.

4 Schéma de la base de données

Le modèle de données repose sur cinq entités principales, reliées par des relations en cardinalités 0,N / 0,1, décrivant la hiérarchie client → dépôt → pull request → fichier → ligne modifiée.



4.1 Détail des entités et de leurs champs

Entité	Champ	Description
CLIENT	num_id (PK)	Identifiant unique du client
	nom	Nom de famille
	prenom	Prénom
	email	Adresse e-mail de connexion
REPO_GIT	rep_id (PK)	Identifiant unique du dépôt
	nom	Nom du dépôt
	url	URL du dépôt sur la plateforme
	plateforme	GitHub / GitLab / Bitbucket
	branche	Branche par défaut suivie
PULL_REQUEST	pull_id (PK)	Identifiant unique de la PR
	commentaire	Commentaire global (verdict) posté sur la PR
	name	Titre de la PR
	date	Date de création / dernière mise à jour
FICHIER	fichier_id (PK)	Identifiant unique du fichier
	status	Statut du fichier dans le diff (ajouté / modifié / supprimé)
	name	Nom du fichier
	relative_path	Chemin relatif dans le dépôt
	date_modification	Date de dernière modification
LIGNE_MODIFIE	ligne_id (PK)	Identifiant unique de la ligne modifiée
	status	Type de modification (ajout / suppression / inchangé)
	commentaire	Commentaire de revue associé à la ligne
	ancienne	Contenu de la ligne avant modification
	nouvelle	Contenu de la ligne après modification
	numero_ligne	Numéro de la ligne dans le fichier

4.2 Relations et cardinalités

Relation	Entre	Cardinalités
POSSEDE	CLIENT ↔ REPO_GIT	(0,N) – (0,N)
CONTIENT	REPO_GIT ↔ PULL_REQUEST	(0,N) – (0,1)
MODIFIE	PULL_REQUEST ↔ FICHIER	(0,N) – (0,N)
APPARTIENT	FICHIER ↔ LIGNE_MODIFIE	(0,N) – (0,1)

Remarque : un CLIENT peut posséder plusieurs REPO_GIT et un REPO_GIT peut être partagé (ex. co-maintenance), un REPO_GIT contient plusieurs PULL_REQUEST, une PULL_REQUEST modifie plusieurs FICHIER, et un FICHIER contient plusieurs LIGNE_MODIFIE annotées.

5 Planning de développement (jusqu'à la bêta)

Le développement est découpé en 6 lots de 2 semaines, aboutissant à une version bêta fonctionnelle de bout en bout (déclenchement automatique, analyses, commentaires sur la PR, dashboard).

Période	Lot	Fonctionnalités développées
Semaines 1–2	Lot 1 — Socle	Mise en place du projet Micronaut/Maven et Svelte/Bootstrap, modélisation et création de la base DuckDB (persistance JDBC), maquettage des écrans (F2, F11, F12), squelette de navigation.
Semaines 3–4	Lot 2 — Intégration Git	F1 (connexion GitHub/GitLab/Bitbucket), F2 (gestion des dépôts suivis), F3 (réception des webhooks + file d'attente). Persistance CLIENT et REPO_GIT en base.
Semaines 5–6	Lot 3 — Analyses statiques	F4 (intégration Checkstyle, PMD, SpotBugs, Error Prone, JaCoCo, JUnit, Javadoc), F5 (exécution en sandbox Linux), F6 (support du fichier <code>.reviewbot.yml</code>).
Semaines 7–8	Lot 4 — Revue LLM & fusion	F7 (revue par LLM sur le diff), F8 (agrégation, déduplication, fusion des résultats statiques + LLM et calcul du verdict global).
Semaines 9–10	Lot 5 — Restitution sur la PR	F9 (commentaire global sur la PR), F10 (commentaires en ligne sur les diffs), F11 (écran de revue de PR complet : Files/Diff/Comments).
Semaines 11–12	Lot 6 — Dashboard & bêta	F12 (dashboard historique), F13 (filtres catégorie/sévérité), F14 (flag humain), tests de bout en bout, corrections de bugs → livraison de la bêta .